RATE MY ESTATE · CHANGE REQUEST · TECHNICAL PLAN

Controlled Locality Selection for Reviews

Replacing free-text location capture with a referenced locality record, seeded from Geoscape G-NAF and shared by the public review form and the Admin Portal dashboard.

REPOSITORY	rate-my-estate-cms
RESOLVES	RME-197, RME-198, RME-199, RME-200 — at root cause, not symptom
RELATED	RME-176 (mandatory review fields)
CHANGE TYPE	Data model change + shared component + data migration
AFFECTED APPS	Laravel 6 backend, React admin portal (`cms/`)
STATUS	Plan for review — build gated on client answers and approved estimate
DATE	17 September 2026

1. Guiding constraint

No new applications, no new libraries, no repository restructure. Every change lands inside conventions the codebase already uses: additive Laravel migrations, commands in `app/Console/Commands`, a thin controller extending `BaseController`, a script in `backend/public/js`, and modules under `cms/src/api` and `cms/src/components/atoms`.

2. Why this is smaller than the ticket implies

An audit of the codebase found that most of the requested locality infrastructure already exists in some form. The plan extends it rather than building alongside it.

`suburbs` **table already exists**	`backend/database/migrations/2020_04_21_063457_create_suburbs_table.php` defines `name`, `postcode` (already a string), `state`, `latitude` and `longitude`, seeded with 15,269 rows from `backend/database/data/au-suburbs.json`. This is the requested locality table, missing only five columns and its indexes.
The eight states are already a table	`review_states` is seeded with exactly the eight states and territories, so the states endpoint is a read of existing data — no new table and no hardcoded list.
State and suburb resolution already exists	`backend/app/Support/StateResolver.php` handles id, name and abbreviation resolution, and `SuburbStateLookup.php` resolves a suburb plus postcode to a state.
The public form already uses select2	`review-step1.blade.php` already loads select2, which supplies the ARIA combobox pattern, keyboard navigation and debounced remote search without any new dependency.
The admin already has a searchable dropdown	`cms/src/components/atoms/ScrollableSelectDropdown.tsx` and `PropertySuburbSelect.tsx` give a working pattern for the new shared control to build on.

3. Two decisions that shape the work

> DECISION 1 — CONFIRMED
>
> **Extend the existing `suburbs` table in place; do not create a second `localities` table.** The ticket's requested fields are added as columns, and `reviews.locality_id` becomes a foreign key to `suburbs.id`. The existing `name` and `state` (full name) columns are left untouched, so `SuburbStateLookup`, `HomeController::getMapData`, `getSuburbsByState` and the property forms continue to work with zero edits. This avoids two overlapping reference tables and a long dual-write transition.

DECISION 2 — CONFIRMED

One API contract, two thin UI implementations. The public review form is Laravel Blade with jQuery and select2; the admin portal is React. A single literally-shared component is not possible without a structural change neither side wants. Instead a reusable select2 module serves the Blade form and a reusable `LocalitySelect` component serves the admin portal — identical behaviour, identical endpoints, each native to its host.

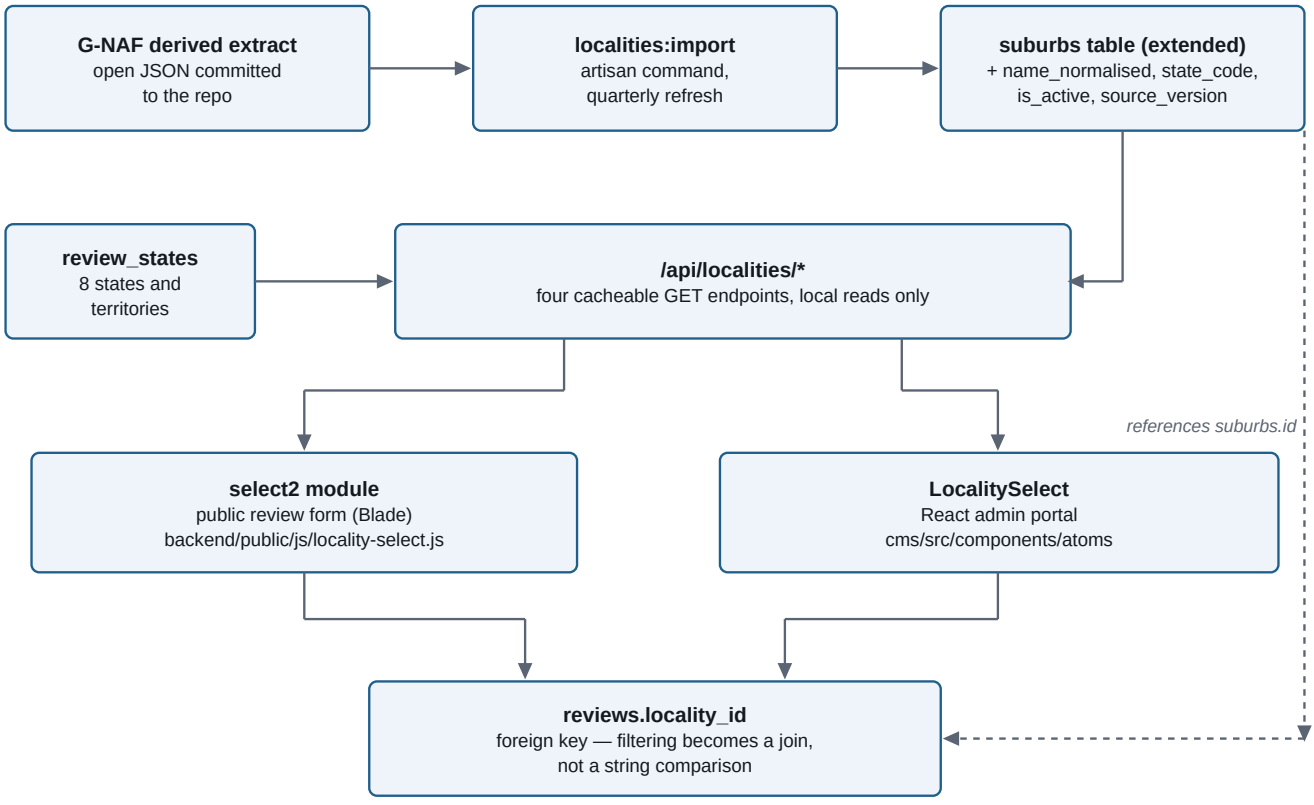
4. Confirmed root cause of RME-200

The Rating Attributes suburb filter cannot work as currently written, and the defect is a data-model problem rather than a coding slip:

- `cms/src/pages/dashboard/shared/useSuburbsByState.ts` puts `suburbs.id` into each dropdown option's value.
- `buildRatingAttributesPayload` in `dashboardUtils.ts` sends that numeric id to the API as `suburb`.
- `Admin/DashboardController.php` then runs `where('suburb', 'LIKE', '%' . $suburb . '%')` against the suburb *name* column.

An id compared against a name can never match. This is precisely why fixing the filters without changing the data model keeps producing regressions, and why storing a foreign key — turning the filter into a join — resolves RME-198 permanently.

5. Target architecture



One seeded reference table, one API contract, two host-native controls, one foreign key on reviews.

6. Implementation phases

PHASE 1 Data model — additive migrations

Two migrations following the existing `add_*_to_*` naming convention.

`add_locality_fields_to_suburbs`

- Add `name_normalised` (string, indexed), `state_code` (string 3, indexed), `is_active` (boolean, default true, indexed), `source_version` (string, nullable) and nullable `timestamps`.
- Make `latitude` and `longitude` nullable — they are currently `NOT NULL`.
- Backfill `name_normalised` and `state_code` from existing rows using `StateResolver`.
- **Deduplicate before indexing.** The existing 15,269 rows may contain duplicates, so delete duplicates keeping the lowest `id`, then add the composite unique index on `(name_normalised, postcode, state_code)` plus an index on `postcode`. This runs before `locality_id` exists, so no review can be orphaned by the cleanup.

`add_locality_id_to_reviews`

- `unsignedBigInteger('locality_id')`, nullable and indexed, with a foreign key to `suburbs.id`. Nullable at database level so existing rows survive; mandatory only in request validation, per the ticket.
- Legacy `state`, `suburb` and `post_code` columns are retained for the migration period.

Model work

- `app/Models/Suburbs.php` — new fillables, `scopeActive`, `scopeInState($code)`, `scopeSearch($q)` with prefix-first ordering, a `display_name` accessor doing title case with `Mc*`, `O'*` and `St` handling, and a static `normalise($name)` that lowercases, strips apostrophes, hyphens and punctuation, and collapses whitespace.
- `app/Models/Review.php` — a `locality()` belongsTo relationship.

TOUCHES 2 new migrations · `Suburbs.php` · `Review.php`

PHASE 2 Repeatable import command

New `app/Console/Commands/ImportLocalities.php` with the signature `localities:import {--file=} {--version=} {--dry-run}`. It is auto-discovered by the existing `$this->load(__DIR__.'/Commands')` call in `Console/Kernel.php`.

Source data is committed to `backend/database/data/localities-gnaf.json`, derived from the `joelkoen/postcodes-au` open extract of G-NAF. Committing it matches the existing `database/data/*.json` convention and means deployments need no outbound network access.

- Restrict to the eight states and territories. The current dataset also contains Christmas Island, Cocos (Keeling) Islands, Norfolk Island and Jervis Bay Territory, none of which have a `review_states` row.
- Exclude non-physical localities (PO Box, large volume receiver, mail centre) on the source type field plus a name guard, so a reviewer can never select a PO Box.
- Read postcodes as strings end to end, so the Northern Territory's `0800` is never silently corrupted to `800`.

- Deduplicate on `(name_normalised, postcode, state_code)`, then upsert: insert new rows, update changed rows, and set `is_active = false` for rows absent from the release. Nothing is ever hard-deleted, so historic reviews keep resolving.
- Stamp `source_version` with the release identifier and print counts of rows added, updated, retired and skipped.

`AuSuburbsSeeder` and `SuburbsSeeder` stay in place for backwards compatibility; the command becomes the maintained path and is documented for the quarterly refresh.

TOUCHES 1 new command · 1 new data file · docs

PHASE 3 Internal API — one new controller

New `app/Http/Controllers/LocalityController.php` extending `BaseController` and returning through the existing `formatSuccessResponse` helper. Routes are registered unauthenticated at the top of `routes/api.php`, alongside the existing `fetch-suburbs` route.

- `GET /api/localities/states` — reads `review_states`.
- `GET /api/localities?state=QLD` — all active localities, alphabetical, wrapped in `Cache::remember` keyed by state and `source_version` to hold the 500ms budget.
- `GET /api/localities/search?state=QLD&q=asc` — minimum two characters, prefix matches ordered ahead of contains matches, limited to 25 results.
- `GET /api/localities/by-postcode?postcode=4007` — all localities sharing that postcode.

Each row returns `{ id, suburb, postcode, state_code, state_name, label, latitude, longitude }`. The `label` is disambiguated server-side — suburb name plus postcode when the name repeats inside a state, plus state code when searching across all states — so neither client has to guess. Responses carry `Cache-Control: public, max-age=86400`. No outbound third-party call is made on any request path.

The existing `/fetch-suburbs`, `/suburbs-by-state` and `/suburbs-by-state-with-postcode` endpoints are left untouched so nothing currently consuming them breaks.

TOUCHES 1 new controller · `routes/api.php`

PHASE 4 Public review form — Blade and select2

New reusable `backend/public/js/locality-select.js` exposing `RmeLocalitySelect.init({ stateEl, suburbEl, postcodeEl, localityIdEl })` and using select2's `ajax.delay: 200` against the search endpoint.

This covers FR-1 through FR-9: suburb disabled until a state is chosen; postcode auto-filled and read-only with a visible override for a single-postcode suburb; postcode promoted to a dropdown with no default when a suburb spans several; typing four valid digits filters the suburb list and sets the state; changing state clears both dependent fields.

Wired into `review-step1.blade.php`, replacing the current inline state and suburb script block, and into `review-step2-1.blade.php`, which today accepts free-text suburb and postcode and is the largest single source of bad data. Both gain a hidden `locality_id` input.

Backend changes in `app/Http/Controllers/ReviewController.php`

- Add `'locality_id' => 'required|exists:suburbs,id'` to `saveReviewStepOne` and `saveReviewStepTwoHomeLand`, and to `Review::$rules` and `$rules_admin`.
- **FR-7 server guard:** re-derive `state`, `suburb` and `post_code` from the resolved locality row and overwrite whatever was posted. The legacy text columns can then never drift out of sync or be spoofed, and an unresolvable combination is rejected rather than stored.
- Extend `applyPropertyLocationToReviewData` to resolve a `locality_id` from the property's suburb, postcode and state.
- Carry `review-locality-id` through the session and copy it in `insertReviewOfNewUser` in `Auth/RegisterController.php`, which currently bypasses `StateResolver` altogether and is a known source of inconsistent rows.

TOUCHES 1 new JS module · 2 Blade views · `ReviewController.php` · `RegisterController.php`

PHASE 5 Admin portal — React

New `cms/src/api/localities/localitiesAPIEndpoints.ts` and `localitiesAPIHandler.ts`, following the established per-domain API module pattern.

New `cms/src/components/atoms/LocalitySelect.tsx` — the shared control covering State, Suburb and Postcode, built on the existing `ScrollableSelectDropdown`. That component needs two additions: an optional external-search mode (`onSearchChange`, so the server drives filtering instead of the current local `includes` filter), and full keyboard and screen-reader support (`role="combobox"`, `aria-activedescendant`, arrow keys, Enter, Escape) to meet the accessibility requirement.

Consumers

- `pages/reviews/ReviewsListPage.tsx` — a filter bar above `DataTable` using `LocalitySelect` plus a single clear-all control (FR-10, FR-13, FR-17), sending `state_code`, `locality_id` and `postcode` alongside the DataTables payload. `getDatatableReview` currently filters on `status` only, so it gains a join on `locality_id` rather than a string comparison (FR-11, FR-12).
- `pages/reviews/ReviewEditPage.tsx` — replace the state `FormControl`, `PropertySuburbSelect` and read-only postcode trio with `LocalitySelect`, submit `locality_id`, and display the resolved state and suburb (FR-14).
- `dashboard/components/RatingAttributesFilter.tsx` and `dashboardUtils.ts` — swap `useSuburbsByState` and `suburbId` for `LocalitySelect`, send `locality_id` and `state_code`, and change `filterRatingChart` to filter on `locality_id`. This is the fix for RME-200 (FR-15).
- `dashboard/components/PropertyReviewsFilter.tsx` — the same control, so both dashboard sections read one source of truth. `applyPropertyLocationFilter` keeps working from names derived off the locality row.

`PropertySuburbSelect` and `usePropertySuburbs` remain in place for the property forms. Migrating those to `LocalitySelect` is a later cleanup, deliberately outside this ticket.

TOUCHES 3 new files · 5 existing components · `Admin/ReviewController.php` · `Admin/DashboardController.php`

PHASE 6 Backfill of existing reviews

New `app/Console/Commands/BackfillReviewLocalities.php` with the signature `reviews:backfill-localities {--dry-run}`.

Matching runs in order: normalised suburb plus state code, then normalised suburb plus postcode, then postcode plus a normalised prefix. `locality_id` is written only on a confident single match — the command never picks the first of several candidates.

Unmatched reviews are written to a CSV exception report using the Maatwebsite Excel dependency already used by `DashboardController::downloadCsv`, on the same S3 or public disk. One admin endpoint returns the download URL so the client can pull the report from the portal and resolve the ambiguous cases themselves.

Legacy text columns are **not** dropped. That becomes a follow-up ticket, gated on the exception list being cleared and client sign-off.

TOUCHES 1 new command · 1 admin endpoint

PHASE 7 Attribution, documentation and tests

- **G-NAF attribution** added to the `btm-footer` block of `resources/views/layouts/app-footer.blade.php`. This is a licence obligation and must ship in the same release as the data, not after it.
- Quarterly refresh runbook added under `docs/`, with a pointer from `README.md`, and the refresh added to the support and maintenance schedule.
- Feature tests under `backend/tests/Feature/` (PHPUnit is already configured) covering the four endpoints, a suburb with multiple postcodes, a duplicate suburb name across states (Richmond), Northern Territory `0800` remaining a string, PO Box exclusion, and submission being blocked without a resolvable locality.

TOUCHES 1 Blade partial · docs · new test files

7. Edge cases and where each is handled

One suburb, several postcodes	No auto-selection in either control; the postcode field becomes a dropdown with no default. The server-side FR-7 guard rejects an unresolved combination.
One postcode, several suburbs	The same guard. A single suburb is never auto-selected when several are valid.
Duplicate name across states (Richmond)	The server-built `label` carries the state code on all-states searches, so the UI never has to construct or guess it.
Duplicate name within one state	The same `label` carries the postcode for disambiguation.

PO Box, large volume receiver, mail centre	Filtered out at import, so a non-physical locality never reaches the database and can never be selected.
Northern Territory leading zeros	`postcode` is already a string column and stays a string through import, API, both clients and storage.
Punctuation (O'Connor, St Kilda, Coffs Harbour)	`Suburbs::normalise()` for matching, so `oconnor` and `st kilda` both return results; `display_name` preserves the display form.
Uppercase source data, names like McKinnon	The `display_name` accessor applies title case with explicit `Mc*`, `O'*` and `St` handling.
Locality retired between releases	`is_active = false`, never a hard delete, so existing reviews continue to resolve and remain filterable.

8. Non-functional coverage

Search within 300ms at p95	Indexed prefix search on `name_normalised` against roughly 16,000 local rows, capped at 25 results.
Full state dropdown within 500ms	`Cache::remember` keyed by state and `source_version`, plus a 24-hour `Cache-Control` header.
No third-party call on the submission path	All four endpoints read the local table only. No outbound HTTP at request time anywhere in the flow.
Keyboard and screen-reader support	select2 supplies the ARIA combobox pattern on the public form; `ScrollableSelectDropdown` is extended with the equivalent roles and key handling for the admin.
Mobile viewport, no horizontal scroll	Stacked layout variant on the shared control; suburb labels truncate rather than overflow.
Attribution live on the site	Phase 7, shipped in the same release as the data.

9. Items needing confirmation before build

SCOPE QUERY RAISED BY THIS AUDIT

FR-16 (dashboard map plotting reviews from locality centroids) has no existing counterpart. The React admin dashboard has no map at all — the only map in the product is the public one served by `HomeController::getMapData`. Adding a dashboard map is net-new work rather than a change to an existing view, and should be priced separately or explicitly confirmed as in scope.

CLIENT QUESTIONS STILL GATING THE BUILD

Per the ticket's own scope note, the four open questions remain unanswered: whether a user may review any valid locality or only one already in the system (recommendation: any valid locality); whether the public front-end review search adopts the same three-field component; whether rural and unincorporated localities are in scope; and whether the client can add a locality manually through the admin portal. The first and third materially affect the import filter, so they should be answered before Phase 2 begins.

10. Out of scope

Street-level address capture or validation; address autocomplete against full G-NAF address records; suburb or postcode boundary polygons; radius search; and any change to review content, rating attributes or the moderation workflow. To be quoted separately if raised.

Incorporates or developed using G-NAF © Geoscape Australia licensed by the Commonwealth of Australia under the Open Geo-coded National Address File (G-NAF) End User Licence Agreement. — the attribution required in the site footer by Phase 7.